Universidad del Valle
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación



Reutilización de código y refactorización de código: Test de software aprobados





El siguiente documento ha sido elaborado tomando fragmentos de documentos previamente publicados, así como imágenes y ejercicios, sobre los cuales se han realizado traducciones libres y adaptaciones con el propósito de desarrollar el material para el curso de Fundamentos de Programación Orientada a Objetos de la Universidad del Valle. Se ha respetado la autoría original de los materiales, y se han proporcionado las referencias correspondientes dentro del documento. El uso de este material se restringe estrictamente al marco académico y formativo de los estudiantes que cursen la asignatura, y no tiene fines comerciales. Queda prohibida su copia o distribución fuera del contexto de dicho curso.

Reutilización de código y refactorización de código:

Test de software aprobados

De acuerdo con [1] la reutilización de código y el refactoring son prácticas fundamentales para mejorar la calidad y la mantenibilidad del software. La reutilización de código permite aprovechar componentes existentes, y utilizar este en múltiples componentes de nuestro programa o en otros proyectos [2], lo cual permite a saber:

- Mantenimiento simplificado: al reducir el código duplicado o innecesario, se hace más fácil de leer y de modificar, lo que facilita realizar actualizaciones o correcciones de errores sin tener que rastrear múltiples partes del código.
- Mejora en la eficiencia y rendimiento: menos código significa que el programa puede ejecutarse más rápido y con menos consumo de memoria, ya que hay menos instrucciones para procesar. Esto puede ser especialmente importante en sistemas con recursos limitados.
- Aplicación de principios de diseño eficientes: la reducción de código se logra aplicando principios como el DRY (Don't Repeat Yourself), el cual promueve la creación de métodos y clases reutilizables, el encapsulamiento y la modularidad. Esto no solo optimiza el tamaño del código, sino que también hace que el sistema sea más fácil de extender y modificar.
- Mejor legibilidad y comprensión: un código más corto y conciso suele ser más fácil de leer y entender, especialmente cuando se aplica una estructura lógica clara. Esto facilita la colaboración entre equipos de desarrollo, ya que otros desarrolladores pueden entender más rápido el propósito y el funcionamiento del código.
- Reducción de errores: con menos código, es menos probable que se introduzcan errores. Además, al centralizar funcionalidades comunes en métodos o clases reutilizables, los errores corregidos en un lugar se reflejan en todas las áreas donde se usa esa funcionalidad, eliminando la necesidad de corregir múltiples instancias.



- Facilidad para realizar pruebas: Un código más compacto y modular permite realizar pruebas unitarias más específicas.

Por otro lado, el refactoring implica modificar la estructura del código sin alterar su comportamiento externo [1]. Distintas técnicas ayudan a reducir la complejidad, aumentar la claridad y facilitar la extensibilidad. Las técnicas permiten que el código sea más legible y modular, facilitando el trabajo colaborativo y la adaptación a requisitos futuros [2].

En el contexto de la programación orientada a objetos y específicamente en el desarrollo en C++, el refactoring es una práctica fundamental para mejorar la estructura interna del código sin alterar el comportamiento del software como tal. Este proceso implica reorganizar y simplificar el código, facilitando su mantenimiento, comprensión y ampliación. Refactorizar permite eliminar duplicidades, mejorar la legibilidad y reducir la complejidad. En el libro [1] se explica cómo el refactoring también puede ser una etapa para implementar patrones de diseño que hacen que el código sea más claro y modular, permitiendo así una adaptación más fácil a futuros cambios y mejoras.

La refactorización hacia patrones representa una técnica en el desarrollo de software orientado a objetos, en la que los patrones de diseño se aplican para optimizar el código existente y facilitar su comprensión y mantenimiento. Donde, un patrón de diseño es una solución reutilizable y probada para problemas comunes de diseño en el desarrollo de software. Estos patrones no son código específico, sino guías o estructuras generales que ayudan a organizar y estructurar el código de manera eficiente, promoviendo la reutilización, flexibilidad y mantenimiento a largo plazo. Cada patrón aborda un problema particular dentro de un contexto y facilita la comunicación entre desarrolladores al proporcionar un lenguaje común para resolver problemas de diseño complejos, este tema se verá más adelante en el curso. Así, este enfoque permite introducir patrones de manera selectiva donde sean verdaderamente beneficiosos, guiando así el proceso de mejora sin imponer estructuras innecesarias. De este modo, la refactorización no solo optimiza el diseño actual sino que también establece un código más flexible y escalable para futuros cambios, promoviendo la claridad en la lógica del programa y la consistencia en la estructura.

Entonces la refactorización se centra en mejorar la claridad del código, reducir su complejidad y simplificar su mantenimiento a largo plazo. Al introducir patrones de diseño, se eliminan las duplicaciones y redundancias que pueden generar problemas de legibilidad y dificultar la modificación o extensión del código. De esta forma, los patrones permiten que el proceso de transformación del código siga un marco estructurado y probado, guiando la reorganización del software hacia una estructura más modular y manejable.

A continuación, se presenta un ejemplo, se toma como base el código disponible en repositorio trabajado en guías anteriores https://github.com/vbucheli/IMC. En el cuál se deben refactorizar los siguientes aspectos:

1. Modificar las clases de la aplicación para que estas tengan una responsabilidad única, de esta manera en siguiente recuadro se presentan cada una de las clase:

```cpp
/*
Información CRC:
Clase: Persona
Responsabilidades:
- tiene la informacion de una persona, para este caso Peso y altura pero podria tener otros campos por ejemplo porcentaje de grasa
- valida y almacena la información para calcular el IMC
Colaboradores:
- No tiene colaboradores directos con otras clases en esta implementación,
 pero podría integrarse con clases que manejen bases de datos o interfaces
 de usuario para expandir su funcionalidad.
*/
#ifndef PERSONA_H
#define PERSONA_H
#include <iostream>
#include <cmath>       // Biblioteca para funciones matemáticas
#include <curl/curl.h> // cURL (Client URL) es una herramienta de línea de comandos y una biblioteca que se utiliza para transferir datos desde o hacia un servidor utilizando una gran variedad de protocolos, como HTTP,
#include <string>
class Persona
{
private:
   double peso;   // Atributo para el peso en kilogramos
   double altura; // Atributo para la altura en metros

public:
   // Constructor para inicializar los atributos
   Persona();
   // Método para traer peso y altura
   double getPeso() const;
   double getAltura() const;
};
#endif // PERSONA_H
```




```cpp
/*
Información CRC:
Clase: IMC
Responsabilidades:
- Calcular el Índice de Masa Corporal (IMC) utilizando el peso y la altura de una persona.
- Calcular el Índice de Masa Corporal (IMC) utilizando dos métodos: una función y una API
- Evaluar el estado físico de la persona con base en su IMC.
Colaboradores:
- Tiene colaboradores directos: Persona
 */
#ifndef IMC_H
#define IMC_H


#include <iostream>
#include <cmath>       // Biblioteca para funciones matemáticas
#include <curl/curl.h> // cURL (Client URL) es una herramienta de línea de comandos y una biblioteca que se utiliza para transferir datos desde o hacia un servidor utilizando una gran variedad de protocolos, como HTTP,
#include <string>
#include "APIClient.h"
class IMC
{
private:
   APIClient apiCliente;       // Instancia del cliente API
public:
   // Constructor para inicializar los atributos
   IMC();
   // Método para calcular el IMC
   double calcularIMC(double peso, double altura) const;
   double calcularIMCAPI(double peso, double altura) const;
   // Método para obtener una evaluación del IMC
   void evaluarIMC(const double &peso, const double &altura) const;
};
#endif // IMC_H
```

```
/*
Información CRC:
Clase: APLICLient
   Responsabilidades:
   - Realizar solicitudes HTTP a la API de cálculo del IMC.
   - Configurar las opciones de solicitud, incluyendo la URL y los encabezados necesarios.
   - Manejar la captura y procesamiento de la respuesta de la API.
   - Manejar errores y excepciones que puedan ocurrir durante las solicitudes.
   Colaboradores:
   - cURL: Utilizado para la comunicación HTTP.
   - nlohmann::json: Utilizado para manipular y analizar datos JSON.
   Autor: Victor Bucheli
   Correo: victor.bucheli@correounivalle.edu.co
   Fecha: Octubre 2024
*/


#ifndef APICLIENT_H
#define APICLIENT_H
#include <string>
class APIClient {
public:
   std::string realizarSolicitud(const std::string &url, const std::string &hd1, const std::string &hd2) const;
};
#endif // APICLIENT_H
```

Recuadro 1. Estructuras de clases, responsabilidad única

2. Uso de funciones: la función calcularIMCAPI() debería devolver una estructura más

3. En la clase Persona hacer validación de entrada: el código no valida la entrada del usuario. Agregar una verificación para asegurarse de que el peso y la altura ingresados sean valores positivos, se modifica el constructor de la clase Persona. Ver recuadro 2

```cpp
Persona::Persona( ) {
double peso=0, altura=0;
// Solicitar y validar la altura
   do {
       std::cout << "Ingrese su altura en metros (positivo): ";
       std::cin >> altura;
       if (altura <= 0) {
           std::cout << "Error: La altura debe ser un valor positivo."
<< std::endl;
       }
   } while (altura <= 0); // Repetir mientras la altura no sea válida
// Solicitar y validar la peso
   do {
       std::cout << "Ingrese su peso en kilogramos (positivo): ";
       std::cin >> peso;
       if (peso <= 0) {
           std::cout << "Error: El peso debe ser un valor positivo."
<< std::endl;
       }
   } while (peso <= 0); // Repetir mientras el peso no sea válido
// Asignar altura y peso validados
this->altura=altura;
this->peso=peso;
}
```

Recuadro 2. Validación de entrada

4. Tener un método separado para procesar el resultado, se modifican los archivos IMC.h y IMC.cpp.

```cpp
double calcularIMCAPI(double peso, double altura) const;
```




```
2   double IMC::calcularIMCAPI(double peso, double altura) const
3   {
4      double retornar=0;
5                                                    Generate Ctrl I
6
7      // Usar std::stringstream para construir la URL dinámicamente
8      std::stringstream url; // Crea un objeto stringstream para
    construir la URL
9      // Inserta el peso y la altura en la URL
10     url << "https://body-mass-index-bmi-
    calculator.p.rapidapi.com/metric?weight=" << peso << "&height=" <<
    altura;
11     std::string hd1="x-rapidapi-key:
    261657ce93msha29aa7438d5a366p1adfa7jsn7bdd387f38d7";
12     std::string hd2="x-rapidapi-host: body-mass-index-bmi-
    calculator.p.rapidapi.com";
13
14
15     // Convierte el contenido del stringstream a un std::string
16     std::string urlStr = url.str();
17
18
19     try {
20
21
22         std::string responseString =
    apiCliente.realizarSolicitud(urlStr, hd1, hd2); // Llama al adaptador
    APIClient
23         // Procesar la respuesta JSON
24         nlohmann::json jsonResponse =
    nlohmann::json::parse(responseString);
25
26         // Verifica si los campos esperados están presentes
27         if (jsonResponse.contains("bmi")) {
28            retornar= jsonResponse["bmi"];
29          return retornar;
30         } else {
31             throw std::runtime_error("Respuesta JSON no contiene los
    campos esperados.");
32         }
33
34
```

```
35      } catch (const std::invalid_argument &e) {
36          std::cerr << "Error al parsear JSON: " << e.what() <<
    std::endl; // Manejo de errores de JSON
37      } catch (const std::runtime_error &e) {
38          std::cerr << "Excepción capturada: " << e.what() << std::endl;
    // Manejo de excepciones
39      } catch (...) {
40          std::cerr << "Error desconocido al procesar la solicitud." <<
    std::endl; // Captura de cualquier otro error
41      }
42      return 9999; // Retorna 9999 en caso de error
43  }
```

Recuadro 3. Modificación los archivos IMC.h y IMC.cpp

5. Método JSON y Dependencia de cUR: el método JSON es redundante ya que calcularIMCAPI() también maneja la respuesta JSON. Se podría integrar esta funcionalidad en el método que realiza la solicitud API. Ya resuelto en líneas anteriores; y la clase Persona depende de cURL directamente. Para una mejor modularidad, podrías considerar utilizar un adaptador o una clase separada que maneje las llamadas a API, una clase llamada APIClient, ver recuadro 1 y 4.




```cpp
std::string APIClient::realizarSolicitud(const std::string &url, const std::string &hd1, const std::string &hd2) const  {
    CURL* hnd = curl_easy_init();
    if (!hnd) {
        throw std::runtime_error("Error al inicializar cURL");
    }

    std::string responseString;
    struct curl_slist* headers = NULL;
    headers = curl_slist_append(headers, hd1.c_str());
    headers = curl_slist_append(headers, hd2.c_str());
    curl_easy_setopt(hnd, CURLOPT_URL, url.c_str());
    curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);

     // Capturar la salida estándar y redirigirla al string
        FILE* fp = fopen("/tmp/imc_response.txt", "wb"); // Archivo temporal
        curl_easy_setopt(hnd, CURLOPT_WRITEDATA, fp);    // Escribe la salida al archivo



```

```
21      // Captura la respuesta
22     // curl_easy_setopt(hnd, CURLOPT_WRITEFUNCTION, NULL);
23      //curl_easy_setopt(hnd, CURLOPT_WRITEDATA, &responseString);
24
25      CURLcode ret = curl_easy_perform(hnd);
26
27      // Manejo de errores
28      if (ret != CURLE_OK) {
29          curl_slist_free_all(headers);
30          curl_easy_cleanup(hnd);
31          throw std::runtime_error("Error en la solicitud cURL: " +
    std::string(curl_easy_strerror(ret)));
32      }
33      else {
34          fclose(fp);  // Cerramos el archivo
35              fp = fopen("/tmp/imc_response.txt", "r"); // Lo abrimos
    para leer
36              if (fp) {
37                  char buffer[1024]; // Buffer temporal para leer
38                  while (fgets(buffer, sizeof(buffer), fp) != NULL) {
39                      responseString += buffer; // Acumulamos en
    responseString
40                  }
41                  fclose(fp);
42              }
43      }
44
45
46      curl_slist_free_all(headers);
47      curl_easy_cleanup(hnd);
48      return responseString; // Retorna la respuesta como cadena
49  }
```

Recuadro 4. Modificación los archivos APIClien.h.,APIClient.cpp y Persona.cpp

6. Manejo de errores: no hay manejo de errores adecuado en caso de que la API devuelva un error. Se debe considerar la implementación de verificaciones para asegurarse de que la respuesta JSON se parsea correctamente, ver recuadro 3.

Para verificar los cambios realizados, clonar el repositorio https://github.com/vbucheli/IMC. e ingresar a la carpeta Refactoring, en ella están todos los códigos refactorizados. Para compilar el código refactorizado utilizar $ make -f Makefile-Refactoring.



Test de software aprobados

En programación orientada a objetos, los tests de software aprobados o *Approval Tests* son un enfoque de verificación en el cual el estado o la salida de una funcionalidad se compara contra un resultado previamente aprobado. En lugar de definir resultados esperados mediante aserciones directas, los tests de aprobación almacenan la salida esperada en un archivo o base de datos, que actúa como referencia para futuras ejecuciones del test. Al ejecutar el test, la salida actual se compara con la salida aprobada; si coinciden, el test pasa. Este enfoque es particularmente útil cuando se trabaja con salidas complejas, como estructuras de datos grandes o salidas visuales, ya que permite gestionar fácilmente los cambios en los resultados esperados sin tener que modificar el código de prueba.

Por otra parte, los test unitarios en programación orientada a objetos son pruebas automáticas diseñadas para validar la funcionalidad específica de una clase o método de manera aislada, asegurando que cada componente funcione de acuerdo a su especificación. En el desarrollo orientado a objetos, los test unitarios ayudan a detectar errores en fases tempranas del ciclo de vida del software, permitiendo verificar que los métodos de una clase produzcan los resultados esperados ante diferentes entradas, especialmente en aquellos métodos que realizan cálculos o manejan lógica compleja. Un test unitario debe cubrir tanto casos normales como extremos, y se ejecuta de forma rápida para facilitar la detección y corrección de errores. Las herramientas de pruebas en C++, como Google Test, permiten estructurar estos test para lograr un seguimiento detallado del estado de cada unidad de código, ayudando a mantener la calidad y robustez del sistema durante su evolución.

En C++ existen varias formas de realizar test unitarios, cada una con sus propias ventajas y limitaciones. Una opción sencilla es el uso de assert, una función estándar de la biblioteca <cassert>, que permite verificar si una condición se cumple; si no, detiene la ejecución y muestra un mensaje de error. Esta forma es rápida de implementar y útil para pruebas simples, pero no ofrece informes detallados ni permite continuar con otras pruebas si alguna falla, por lo que es menos ideal para proyectos grandes. Otra opción es Google Test, una biblioteca robusta que permite escribir pruebas organizadas en casos y grupos con macros como TEST(). Google Test proporciona un informe detallado sobre el éxito o fallo de cada prueba, soporta casos de prueba parametrizados y permite verificar condiciones de manera flexible; sin embargo, requiere configuración adicional, especialmente si se usa junto con herramientas como CMake. Por último, Catch2 es una alternativa similar a Google Test, pero más ligera y con una sintaxis que utiliza REQUIRE() y SECTION() para organizar pruebas de forma intuitiva. Catch2 es fácil de integrar en un solo archivo y, aunque puede ser más limitada en comparación con Google Test en proyectos complejos, su simplicidad y flexibilidad la hacen una opción popular para pruebas medianas o pequeñas. En el curso trabajaremos con test unitarios y utilizaremos assert, try y google test.



En las siguientes líneas se presentan pruebas aplicadas al proyecto IMC refactorizado. A continuación, se presentan ejemplos de tests unitarios, se toma como base el código disponible en repositorio trabajado en guías anteriores https://github.com/vbucheli/IMC. Las pruebas del proyecto se pueden verificar en el mismo repositorio en la carpeta Tests, y se puede compilar: $make -f Makefile-Test

1. Pruebas assert con valores idénticos: en el siguiente recuadro, se crea un objeto de la clase `Persona` llamado `persona`, inicializándolo con un peso de 74 kilogramos y una altura de 170 centímetros. Posteriormente, se utilizan dos afirmaciones (`assert`) para validar que los valores asignados al objeto son correctos. La primera afirmación comprueba que el peso del objeto `persona` es efectivamente 74.0, asegurando que el constructor ha funcionado correctamente y que el valor se ha almacenado como se esperaba. La segunda afirmación verifica que la altura es 170.0, también confirmando que el método `getAltura()` retorna el valor esperado. Estas verificaciones son esenciales para asegurar la integridad de los datos en el objeto y el correcto funcionamiento de los métodos de la clase.

```
/*
   Proyecto: IMC - Test
   Curso: Fundamentos de Programación Orientada a Objetos (FPOO)
   Universidad del Valle
   Descripción:
   Este proyecto permite trabajar con:
   Uso de clases
   Uso de bibliotecas
   Uso de APIs
   Reutilización de código
   Software modular y compatibles con otro software
   Autor: Victor Bucheli
   Correo: victor.bucheli@correounivalle.edu.co
   Fecha: Octubre 2024
*/
#include <iostream>
#include <curl/curl.h>
#include <cmath> // Biblioteca para funciones matemáticas
#include "Persona.h"
#include "IMC.h"
#include <cassert>
int main()
{
```

```
24      // Crear un objeto de la clase Persona
25      Persona persona(74,170);
26       assert(persona.getPeso( ) == 74.0);   // Verifica que el peso es 74
27       assert(persona.getAltura( ) == 170.0); // Verifica que la altura es
    170
28      return 0;
29  }
```

Recuadro 5. Test con assert

2. Pruebas assert con valores aproximados: la línea de código que realiza la prueba de cálculo del Índice de Masa Corporal (IMC) utiliza la función `assert` para validar que el resultado del método `calcularIMC`, al que se le pasan como argumentos la altura y el peso de un objeto `persona`, se aproxima al valor esperado de `0.0310446`. Se utiliza `std::abs()` para calcular la diferencia absoluta entre el resultado obtenido y el valor esperado, permitiendo así manejar cualquier variación en el cálculo que pueda surgir debido a la precisión de los números de punto flotante. La condición `< 0.01` establece un margen de tolerancia, indicando que el resultado se considera correcto siempre que la diferencia entre el IMC calculado y el valor esperado esté dentro de 0.01. Esta técnica es fundamental en las pruebas unitarias, ya que asegura que los cálculos se realicen de manera precisa y dentro de un rango aceptable, garantizando la fiabilidad de la funcionalidad de la clase `Persona`.

```
1   int main( )
2   {
3   assert(std::abs(calculadora.calcularIMC(persona.getAltura( ),persona.get
    Peso( )) - 0.0310446) < 0.01); // Verifica que el IMC es correcto
4      std::cout << "Todas las pruebas pasaron correctamente.\n";
5      return 0;
6   }
```

Recuadro 6. Test con assert valores aproximados

3. Pruebas try y catch: para evaluar `APIClient`, la cual realiza solicitudes HTTP a una API externa para calcular el Índice de Masa Corporal (IMC). Primero, se crea una prueba que verifica el comportamiento de la función `realizarSolicitud` con una URL válida, que se construye dinámicamente usando un `std::stringstream`. Se define una solicitud a la API que incluye parámetros para peso y altura, y se ejecuta la solicitud. La prueba utiliza `assert` para confirmar que la respuesta no esté vacía, lo que indica que la API respondió correctamente. A continuación, se incluye una prueba teórica para manejar un error en la inicialización de cURL, aunque no se puede simular fácilmente en este contexto; se lanza una excepción esperada para verificar que el mensaje sea el correcto. Luego, se simula un caso de error al pasar una URL no válida, donde se espera que la función también lance una excepción, y se verifica que el mensaje de error contenga la frase `"Error en la solicitud cURL"`.

Finalmente, si todas las pruebas se ejecutan correctamente, se imprime un mensaje de éxito, indicando que el código de la clase `APIClient` maneja adecuadamente tanto las solicitudes exitosas como los errores esperados, ver el siguiente recuadro.

```
int main() {
   APIClient apiClient;


   // Prueba de solicitud válida
   std::stringstream url; // Crea un objeto stringstream para construir la URL
   url << "https://body-mass-index-bmi-calculator.p.rapidapi.com/metric?weight=74&height=171";
   std::string urlString=url.str();
   std::string hd1="x-rapidapi-key: 261657ce93msha29aa7438d5a366p1adfa7jsn7bdd387f38d7";
   std::string hd2="x-rapidapi-host: body-mass-index-bmi-calculator.p.rapidapi.com";


   // Para esta prueba, asumimos que la API está disponible y responderá correctamente
   // **Nota:** En un entorno real, deberías simular la respuesta de la API o usar un entorno de pruebas
   std::string response = apiClient.realizarSolicitud(urlString, hd1, hd2);
```

```
16      assert(!response.empty()); // Verifica que la respuesta no esté
    vacía
17
18
19      // Prueba de manejo de error al inicializar cURL
20      try {
21          // Se puede simular un error creando un objeto APIClient que
    falle al inicializar cURL
22          // Aquí, no hay un método para forzar un fallo, así que esta
    prueba es teórica
23          // assert(apiClient.realizarSolicitud("malformed_url", header1,
    header2)); // Esto debería fallar
24      } catch (const std::runtime_error &e) {
25          // Verifica que se lanza una excepción con un mensaje específico
26          assert(std::string(e.what()) == "Error al inicializar cURL");
27      }
28
29
30      // Prueba de manejo de error en la solicitud cURL
31      // Aquí deberías proporcionar un URL que sepas que fallará, como un
    dominio no existente
32      try {
33          std::string invalidUrl = "http://invalido.com"; // Asegúrate de
    que esta URL falle
34          apiClient.realizarSolicitud(invalidUrl, hd1, hd2);
35      } catch (const std::runtime_error &e) {
36          // Verifica que se lanza una excepción con un mensaje específico
37          assert(std::string(e.what()).find("Error en la solicitud
    cURL:") != std::string::npos);
38      }
39
40
41      // Mensaje de éxito si todas las pruebas pasan
42      std::cout << "Todas las pruebas pasaron correctamente.\n";
43      return 0;
44  }
```

Recuadro 7. Test con try y catch

Pruebas unitarias con google tests

Los tests unitarios son una parte esencial del desarrollo de software, ya que permiten verificar el correcto funcionamiento de pequeñas unidades de código, como funciones o métodos, de manera aislada. Google Test es un popular marco de pruebas en C++ que facilita la creación y ejecución de estos tests. Al utilizar Google Test, los desarrolladores



pueden definir una serie de pruebas automatizadas que comprueban el comportamiento esperado de su código bajo diversas condiciones. A través de instrucciones como `EXPECT_EQ` y `EXPECT_NEAR`, es posible comparar los resultados reales con los resultados esperados, identificando rápidamente errores o comportamientos inesperados. Además, Google Test proporciona una interfaz amigable y un informe detallado sobre los resultados de las pruebas, lo que facilita el proceso de depuración y garantiza la calidad del software a lo largo de su desarrollo. A continuación, se presentan ejemplos de google tests unitarios, se toma como base el código disponible en repositorio trabajado en guías anteriores https://github.com/vbucheli/IMC. Las pruebas del proyecto se pueden verificar en el mismo repositorio en la carpeta IMC-tests, y se puede compilar como se explica en esta guía más adelante.

Vamos a tomar el código de IMC para utilizar pruebas unitarias con google test, así se implementan pruebas unitarias utilizando el marco de trabajo Google Test para la clase `IMC`, que calcula el Índice de Masa Corporal (IMC) y evalúa su estado. Se definen dos pruebas principales: la primera, `CalcularIMC`, verifica que el método `calcularIMC` de la clase `IMC` devuelva resultados cercanos a los valores esperados, utilizando la función `EXPECT_NEAR` para asegurar que la diferencia entre el valor calculado y el esperado esté dentro de un margen de error de 0.01. Por ejemplo, al ingresar un peso de 70 kg y una altura de 1.75 m, se espera que el IMC calculado sea aproximadamente 22.86. La segunda prueba, `EvaluarIMC`, utiliza la función `EXPECT_EQ` para comprobar que el método `evaluarIMC` retorne la clasificación correcta del estado del peso según diferentes combinaciones de peso y altura. Se verifica que el método clasifique adecuadamente los casos de "por debajo del peso normal", "peso normal", "sobrepeso" y "obesidad". Finalmente, la función `main` inicializa Google Test y ejecuta todas las pruebas definidas, asegurando que cualquier error en la lógica de la clase `IMC` se detecte durante el proceso de desarrollo, ver el siguiente recuadro. El código está disponible en el repositorio https://github.com/vbucheli/IMC. Las pruebas del proyecto se pueden verificar en el mismo repositorio en la carpeta IMC-tests.

```cpp
#include <gtest/gtest.h>
#include "../IMC.h"


TEST(IMCTest, CalcularIMC) {
    IMC imc;
    EXPECT_NEAR(imc.calcularIMC(70, 1.75), 22.86, 0.01);  // Prueba de cálculo de IMC
    EXPECT_NEAR(imc.calcularIMC(60, 1.60), 23.44, 0.01);  // Otro ejemplo de cálculo
}


TEST(IMCTest, EvaluarIMC) {
    IMC imc;
    EXPECT_EQ(imc.evaluarIMC(50, 1.75), "Por debajo del peso normal.");
    EXPECT_EQ(imc.evaluarIMC(70, 1.75), "Usted tiene un peso normal.");
    EXPECT_EQ(imc.evaluarIMC(85, 1.75), "Usted tiene sobrepeso.");
    EXPECT_EQ(imc.evaluarIMC(100, 1.75), "Usted tiene obesidad.");
}


// Función main para ejecutar todas las pruebas
int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

Recuadro 8. Pruebas con google test

Los test debe estar en una carpeta tests dentro del proyecto es decir el archivo anterior debe estar en la ruta (test/test_IMC.cpp). De igual manera para ejecutar las pruebas con google test es necesario hacer un archivo CMakeLists.txt, en el siguiente recuadro se muestra el archivo. El archivo `CMakeLists.txt` es un componente clave en la configuración de proyectos de software que utilizan CMake como sistema de construcción. Este archivo define las propiedades del proyecto, como su nombre, la versión mínima requerida de CMake y los estándares de lenguaje C++ a utilizar. Además, especifica los archivos fuente que componen el proyecto y establece las dependencias necesarias, como bibliotecas externas (por ejemplo, Google Test para pruebas unitarias). Al incluir comandos como `add_library` y `add_executable`, `CMakeLists.txt` organiza cómo se compilan y enlazan los diferentes componentes del proyecto, facilitando así el proceso de construcción y asegurando que todos los elementos se integren correctamente.

```
cmake_minimum_required(VERSION 3.10)
project(IMC-tests)
```

```
set(CMAKE_CXX_STANDARD 17)

# Agrega Google Test como subdirectorio
add_subdirectory(googletest)

# Archivos fuente
add_library(imc_lib IMC.cpp)

# Configuración de Google Test
enable_testing()
find_package(GTest REQUIRED)
include_directories(${GTEST_INCLUDE_DIRS})

# Agrega el ejecutable para las pruebas
add_executable(test_imc test/test_IMC.cpp)

# Incluye los encabezados de Google Test
include_directories(${gtest_SOURCE_DIR}/include)

# Enlaza las bibliotecas necesarias
target_link_libraries(test_imc imc_lib ${GTEST_LIBRARIES} pthread)

# Agrega un comando para ejecutar las pruebas
add_test(NAME test_imc COMMAND test_imc)
```

Recuadro 9. Archivo CMakeLists

Para compilar y ejecutar las pruebas en un proyecto que utiliza CMake y Google Test, se siguen varios pasos clave. Primero, se debe crear un directorio de compilación, generalmente llamado `build`, donde se generarán los archivos necesarios. Desde la terminal, se navega a este directorio y se ejecuta el comando `cmake ..` para configurar el proyecto, que lee el archivo `CMakeLists.txt` y genera el sistema de construcción correspondiente. A continuación, se utiliza el comando `make` para compilar el proyecto, lo que compila tanto la biblioteca de la aplicación como los ejecutables de prueba. Finalmente, para ejecutar las pruebas, se puede usar el comando `ctest` en el directorio de compilación, que invoca el ejecutable de pruebas generado, o ejecutar directamente el ejecutable de pruebas para ver los resultados detallados de cada prueba. Estos pasos permiten verificar el correcto funcionamiento del código mediante pruebas automatizadas. En el siguiente recuadro se presentan los comandos.

```
En la carpeta de tu proyecto
$ mkdir build && cd build && cmake .. && make && ./test_imc
```

Recuadro 9. Comandos compilación pruebas

Google Test facilita la implementación y ejecución de pruebas unitarias. Una vez escritas, se compilan junto con el proyecto, y se ejecutan mediante comandos específicos. Durante la ejecución, el framework proporciona una salida que detalla el resultado de cada prueba: si pasó o falló, junto con información adicional sobre errores o fallos en caso de que ocurrieran. Esta salida final es crucial, ya que permite a los desarrolladores identificar y corregir problemas en el código, garantizando que cada componente funcione como se espera antes de su integración en el sistema más amplio. Al finalizar las pruebas, se generan informes que resumen el porcentaje de pruebas pasadas y fallidas, lo que ayuda a evaluar la estabilidad y fiabilidad del software, ver el siguiente recuadro.

```
@vbucheli → /workspaces/IMC/IMC-tests/build (main) $ ./test_imc
[==========] Running 2 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 2 tests from IMCTest
[ RUN      ] IMCTest.CalcularIMC
[       OK ] IMCTest.CalcularIMC (0 ms)
[ RUN      ] IMCTest.EvaluarIMC
Su Índice de Masa Corporal (IMC) es: 16.3265
Su Índice de Masa Corporal (IMC) es: 22.8571
Su Índice de Masa Corporal (IMC) es: 27.7551
Su Índice de Masa Corporal (IMC) es: 32.6531
[       OK ] IMCTest.EvaluarIMC (0 ms)
[----------] 2 tests from IMCTest (0 ms total)

[----------] Global test environment tear-down
[==========] 2 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 2 tests.
```

Recuadro 10. salidas pruebas google test

Referencias

[1] J. Kerievsky, *Refactoring to patterns*. en Addison-Wesley signature series. Boston: Addison-Wesley, 2005.

[2] M. Fowler, *Refactoring: improving the design of existing code*, Second edition. en Addison-Wesley signature series. Boston: Addison-Wesley, 2019.